

Shipping document verification

From email inbox to discrepancy report

Context

A shipping operations team receives different kinds of messages in the same inbox: requests to check documents, prepare new shipping instructions, answer invoice questions, and share operational updates. Spam arrives alongside them.

For a document-checking request, the team compares a **Shipping Instruction (SI)**, which contains the intended shipment details, with a **draft Bill of Lading (BL)**. The SI is the reference for this check. The goal is to catch incorrect details before the draft is finalized.

The problems

- **Finding the right emails takes time.** Staff must read each message and decide what action it needs. A document request that is overlooked never reaches the checking step.
- **Manual comparison is repetitive and easy to get wrong.** Names, ports, quantities, and weight must be checked across two documents. A missed discrepancy can lead to corrections, delays, and additional work.
- **The same information can look different.** One document may say “Port of Loading” while the other says “Load Port.” The system needs to recognize that these refer to the same field.

What the system should be able to do

Starting from the inbox, the system should produce a clear result for each email. How you design the workflow is up to you, but it should generally be able to:

Capability	What it means
Classify	Tell the different kinds of messages apart, including document-comparison requests, new SI requests, invoice queries, general messages, and spam.
Extract data	For comparison requests, read the SI and BL attachments and identify the corresponding shipment fields.
Compare	Check the values and surface any mismatched fields, showing the SI and BL values side by side.
Ask for help	When it cannot complete the task on its own, escalate to a person (human in the loop) with the relevant context, rather than guessing or failing silently.

The starting version uses JSON email records and plain-text attachments. Other email categories only need to be classified; only document-comparison requests continue to the checking step. The approach used to achieve these capabilities is left to the participant.

Expected result and extensions

What the comparison covers

Check seven fields: **shipper, consignee, notify party, port of loading, port of discharge, container count, and gross weight in kilograms.**

The report should make it easy to see which email was checked, whether a mismatch was found, and exactly what needs attention. If all seven fields match, report “No mismatch detected.”

Example

The SI lists 3 containers and 22,000 kg. The BL lists 4 containers and 22,000 kg. If the other fields agree, flag only the container count and show **SI: 3 / BL: 4**.

Advanced stage

Classifying emails, extracting fields from plain text, and comparing values are the **basic, common** expectations. Once that works, we encourage you to go further and attempt a more advanced solution using the **sample data we provide**, which includes more realistic documents and harder decisions.

Advanced challenge	What changes
PDF and Word attachments	Replace plain-text attachments with PDFs and Word documents. You need to extract information from tables and different page layouts.
Scanned documents	Use image-only PDFs or scanned pages. You can use optical character recognition (OCR), a vision-capable LLM, or both to read and compare the content.
Messier inputs	Introduce varied field labels, formatting differences, misleading email subjects, or missing attachments. The system must distinguish a real discrepancy from a reading or formatting issue.
Reliability and human review	When a document is unreadable, a required value is missing, or the result is uncertain, send the case for review with the source evidence and reason. Let a person confirm or correct it, then update the report. Handle processing failures visibly and allow retries.

Accuracy means identifying the right requests and the right discrepancies without creating false alarms. The **reliability** challenge considers what happens when the system cannot make a dependable decision, including when human input is needed.

The basic use case remains the same at every stage: find the document request, compare the shipment details, and explain any mismatch. The advanced stage is where you can stand out by handling the harder, more realistic sample data.

Working with the data

The dataset contains inbox records in JSON, together with the SI and BL attachments referenced by those emails. The answer key is not included. You can check your result using the self-evaluation endpoint described on the next page.

Two ways to access the data

Option	How you use it
Static bundle	A ZIP file containing inbox/ , attachments/ , sample_submission.json , and a helper file called loader.py . Extract the ZIP and read the files directly. No service needs to be started.
Local server (Docker)	Run docker compose up --build to access the same dataset over HTTP at http://localhost:8080 . No database or additional setup is required.

The loader

The included **loader.py** provides the same interface for both options. Point it to either the extracted data folder or the local server:

Example

```
from loader import Inbox

inbox = Inbox("data") — or Inbox("http://localhost:8080")

for email in inbox: reads each email record, while inbox.read_text(path) returns the
text from an SI or BL attachment.
```

Start with one email and its two attachments so you can see how the records are connected. The participant guide included with the data explains the access options and the fields in detail.

Evaluating your own output

How you design your system and what it produces internally is entirely up to you. To help you gauge how well it is doing, the local server includes an optional self-evaluation endpoint. Submit your result and the server compares it with a private reference set, then returns a scoreboard. The reference answers are not included in the response.

Formatting your output for the self-evaluation

The self-evaluation only needs your output in one agreed shape so it can be read automatically: one JSON object keyed by **email_id**, following the format in **sample_submission.json**. Include every email in the dataset, and for a document-comparison request report its category, whether a mismatch was found, and the fields that differ. This shape is only required *if* you want to use the self-evaluation — it is not a constraint on how your system works internally.

How to run it

With the local server running, send your formatted output to `POST /submit` or call `inbox.submit(...)` through the loader. The response contains the evaluation result without exposing the reference answers.

How to use the result

The scoreboard is meant to help you find problems while you build. It is not the final assessment and does not cover every part of a good solution.

- Check where your system classified an email incorrectly or missed a document mismatch.
- Review cases where the input was incomplete or uncertain. A score cannot fully assess whether the system asked for human review at the right time or provided enough context.
- If your result differs from the reference, check the source documents before changing it. If your decision is reasonable, record the reason.

You can submit your output as often as needed while developing. Use the result to improve accuracy, and separately test how your system behaves when information is missing, unclear, or unreadable.